

SpectraGit — Guia de Instalação Self-Hosted

Versão do documento: 1.0
Última atualização: 2026-05-03

1. O que é Self-Hosted?

O SpectraGit pode operar em dois modos:

Modo	Descrição
SaaS	Gerenciado pelo time de engenharia via código e infraestrutura. Não possui painel administrativo nem setup wizard.
Self-Hosted	Você instala e gerencia sua própria instância. Inclui o Setup Wizard (configuração inicial) e o Admin Panel (gerenciamento de usuários, configurações da instância e health check).

Este guia ensina a configurar e executar o SpectraGit no modo **Self-Hosted**.

2. Pré-requisitos

Hardware mínimo

- CPU:** 2 vCPUs
- RAM:** 4 GB
- Disco:** 20 GB (SSD recomendado)
- Rede:** Portas 80 (HTTP), 443 (HTTPS) e 2222 (Git SSH) liberadas

Software necessário

- Docker Engine 24+
- Docker Compose v2.20+
- Git (para clonar o repositório)
- OpenSSL (para gerar secrets JWT)

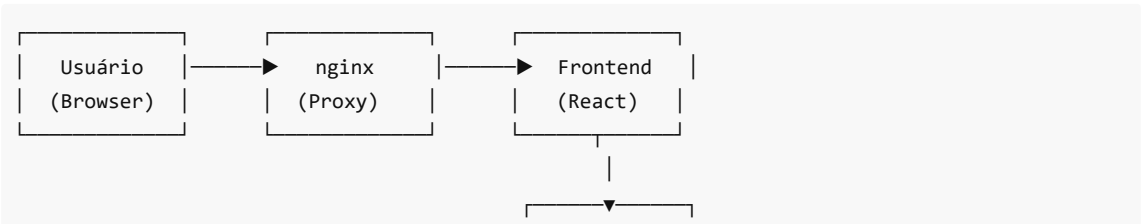
Serviços externos

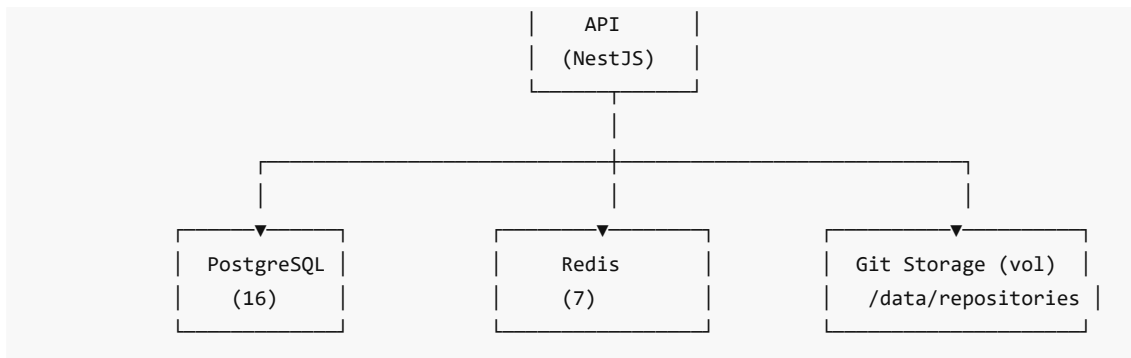
Você precisará configurar pelo menos **um** provedor OAuth para permitir login:

- GitHub OAuth App** ([criar aqui](#))
- Google OAuth 2.0** ([Google Cloud Console](#))

Nota: O login por senha local não está disponível no momento. Toda autenticação web passa por OAuth.

3. Estrutura de Deploy





4. Passo a Passo de Instalação

4.1. Clone o repositório

```
git clone https://github.com/seu-org/spectragit.git
cd spectragit
```

4.2. Configure as variáveis de ambiente

Copie o arquivo de exemplo:

```
cp .env.example .env
```

Edite o arquivo `.env` e preencha os valores obrigatórios:

```
# — Database —————
POSTGRES_USER=spectragit
POSTGRES_PASSWORD=GereUmaSenhaForteAqui123!
POSTGRES_DB=spectragit
DATABASE_URL=postgresql://spectragit:GereUmaSenhaForteAqui123!@postgres:5432/spectragit?
schema=public

# — JWT (mínimo 32 caracteres, use o comando abaixo) —————
# openssl rand -hex 48
JWT_SECRET= cole_aqui_o_resultado_do_comando_acima
JWT_REFRESH_SECRET= cole_aqui_outro_resultado_do_comando_acima

# — Deployment Mode (obrigatório para self-hosted) —————
DEPLOYMENT_MODE=self-hosted

# — URLs —————
# Substitua pelo seu domínio real
FRONTEND_URL=https://git.suaempresa.com
CORS_ORIGINS=https://git.suaempresa.com

# — OAuth: GitHub —————
# Crie um app em https://github.com/settings/applications/new
# Homepage URL: https://git.suaempresa.com
```

```
# Callback URL:      https://git.suaempresa.com/api/v1/auth/oauth/github/callback
GITHUB_CLIENT_ID=seu_client_id_aqui
GITHUB_CLIENT_SECRET=seu_client_secret_aqui
GITHUB_CALLBACK_URL=https://git.suaempresa.com/api/v1/auth/oauth/github/callback

# — OAuth: Google (opcional) —————
GOOGLE_CLIENT_ID=placeholder
GOOGLE_CLIENT_SECRET=placeholder
GOOGLE_CALLBACK_URL=https://git.suaempresa.com/api/v1/auth/oauth/google/callback

# — S3 / Object Storage (opcional, para assets de releases) —————
S3_ENDPOINT=https://nbg1.your-objectstorage.com
S3_REGION=nbg1
S3_BUCKET=seu-bucket
S3_ACCESS_KEY_ID=sua_access_key
S3_SECRET_ACCESS_KEY=sua_secret_key

# — SSH Git —————
SSH_PORT=2222
```

Importante: `DEPLOYMENT_MODE` deve ser `self-hosted`. Se definido como `saas`, o painel administrativo e o setup wizard serão desativados.

4.3. Inicie os containers

```
docker compose up --build -d
```

Aguarde a inicialização completa (pode levar 1–2 minutos no primeiro build).

Verifique se os serviços estão saudáveis:

```
docker compose ps
```

Você deve ver `healthy` em `postgres`, `redis`, `api` e `frontend`.

4.4. Execute as migrations do banco

Se não estiverem rodando automaticamente:

```
docker compose exec api npx prisma migrate deploy
```

5. Primeiro Acesso — Setup Wizard

Com o modo `self-hosted`, ao acessar a URL da instância pela primeira vez, você será redirecionado automaticamente para:

```
https://git.suaempresa.com/setup
```

O **Setup Wizard** possui 3 etapas:

Etapa 1: Configuração da Instância

- **Instance name:** nome público da sua instância (ex: `Git da Empresa`)
- **Public base URL:** o domínio completo (ex: `https://git.suaempresa.com`)
- **Brand color:** cor hexadecimal da marca (ex: `#7C3AED`)

Etapa 2: Criar Conta Administrador

- **Username:** nome de usuário do primeiro admin (ex: `admin`)
- **Email:** email do administrador (deve ser o mesmo email da conta OAuth que você usará para fazer login)
- **Password:** senha forte (mínimo 8 caracteres)

Atenção: A senha criada aqui é armazenada no banco, mas o login web usa **OAuth**. Use o mesmo email do setup ao fazer login via GitHub/Google para que a conta seja vinculada automaticamente.

Etapa 3: Conclusão

Após a conclusão, clique em **"Go to sign in"** e faça login com o provedor OAuth configurado.

6. Acessando o Painel Administrativo

Após logado como administrador, acesse:

```
https://git.suaempresa.com/admin
```

Funcionalidades disponíveis

Aba	Descrição
Overview	Dashboard com resumo da instância e links rápidos
Users	Lista todos os usuários. Permite promover a <code>SYSTEM_ADMIN</code> ou desabilitar contas
Settings	Edita nome da instância, cor, logo, URL base e configurações SMTP
Health	Status do sistema: banco de dados, storage Git, uptime e memória

Como promover um usuário a administrador

1. Vá em **Admin** → **Users**
2. Encontre o usuário na lista
3. Clique no botão de alternar role (ícone de escudo)
4. O usuário passa de `User` para `Admin` (`SYSTEM_ADMIN`)

7. Configuração Avançada

7.1. Configurar SMTP (para notificações por email)

No painel administrativo, vá em **Admin** → **Settings** e preencha:

- **SMTP Host:** seu servidor SMTP (ex: `smtp.gmail.com`)
- **SMTP Port:** porta (ex: `587`)
- **SMTP User:** usuário de autenticação
- **SMTP From Email:** email de envio (ex: `noreply@suaempresa.com`)

- **SMTP Password:** senha ou app-specific password

Clique em **Save**.

7.2. Configurar HTTPS/TLS (Produção)

Para produção, utilize um reverse proxy com certificados SSL. Recomendamos:

Opção A: Traefik (automático com Let's Encrypt)

```
# docker-compose.override.yml
services:
  traefik:
    image: traefik:v3.1
    command:
      - --providers.docker=true
      - --entrypoints.web.address=:80
      - --entrypoints.websecure.address=:443
      - --certificatesresolvers.letsencrypt.acme.tlschallenge=true
      - --certificatesresolvers.letsencrypt.acme.email=admin@suaempresa.com
      - --certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./letsencrypt:/letsencrypt

  nginx:
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.spectragit.rule=Host(`git.suaempresa.com`)"
      - "traefik.http.routers.spectragit.tls.certresolver=letsencrypt"
      - "traefik.http.routers.spectragit.entrypoints=websecure"
```

Opção B: nginx com certificados manuais

Edite `nginx/nginx.conf` para incluir o bloco `ssl` :

```
server {
    listen 443 ssl;
    server_name git.suaempresa.com;

    ssl_certificate      /etc/nginx/ssl/cert.pem;
    ssl_certificate_key  /etc/nginx/ssl/key.pem;

    location / {
        proxy_pass http://frontend:80;
    }

    location /api/ {
        proxy_pass http://api:3000/;
    }
}
```

```
}  
}
```

7.3. Backup do Banco de Dados

Crie um script de backup diário:

```
#!/bin/bash  
# backup.sh  
BACKUP_DIR="/backups/spectragit"  
DATE=$(date +%Y%m%d_%H%M%S)  
docker compose exec -T postgres pg_dump -U spectragit spectragit >  
"$BACKUP_DIR/db_$DATE.sql"  
# Mantém apenas os últimos 7 backups  
ls -t $BACKUP_DIR/db_*.sql | tail -n +8 | xargs -r rm
```

Adicione ao crontab:

```
0 2 * * * /caminho/para/backup.sh
```

7.4. Backup dos Repositórios Git

Os repositórios ficam no volume Docker `git_storage` . Para backup:

```
docker run --rm -v spectragit_git_storage:/data -v /backups:/backups alpine tar czf  
/backups/git_storage_$(date +%Y%m%d).tar.gz -C /data .
```

8. Atualizando a Instância

```
# Puxe as atualizações  
git pull origin main  
  
# Reconstrua e reinicie os containers  
docker compose pull  
docker compose up --build -d  
  
# Execute migrations se necessário  
docker compose exec api npx prisma migrate deploy
```

Sempre leia o `CHANGELOG.md` antes de atualizar em produção.

9. Troubleshooting

Problema: Setup Wizard não aparece

Causa: `DEPLOYMENT_MODE` pode estar definido como `saas` .

Solução:

```
docker compose exec api env | grep DEPLOYMENT_MODE
# Deve retornar: DEPLOYMENT_MODE=self-hosted
```

Se estiver errado, edite o `.env` e reinicie:

```
docker compose up -d
```

Problema: Login OAuth falha

Causa: URL de callback incorreta ou OAuth app mal configurado.

Solução:

- Verifique se `GITHUB_CALLBACK_URL` ou `GOOGLE_CALLBACK_URL` corresponde exatamente ao registrado no provedor OAuth
- Verifique se `FRONTEND_URL` está correto
- Verifique os logs: `docker compose logs api | grep oauth`

Problema: Cannot find user após login OAuth

Causa: O email da conta OAuth é diferente do email cadastrado no setup.

Solução: O SpectraGit vincula contas OAuth por email. Certifique-se de usar o mesmo email no setup wizard e na conta OAuth. Se necessário, corrija o email diretamente no banco:

```
docker compose exec postgres psql -U spectragit -d spectragit -c "UPDATE users SET email = 'novo@email.com' WHERE username = 'admin';"
```

Problema: Erro 404 em `/admin`

Causa: Usuário logado não possui `systemRole = SYSTEM_ADMIN`.

Solução: Verifique no banco:

```
docker compose exec postgres psql -U spectragit -d spectragit -c "SELECT username, system_role FROM users;"
```

E atualize se necessário:

```
docker compose exec postgres psql -U spectragit -d spectragit -c "UPDATE users SET system_role = 'SYSTEM_ADMIN' WHERE username = 'admin';"
```

Problema: Git SSH não conecta na porta 2222

Causa: Firewall bloqueando a porta ou mapeamento incorreto.

Solução:

```
# Verifique se a porta está exposta
docker compose ps
```

```
# Teste a conectividade
telnet git.suaempresa.com 2222

# Verifique logs do SSH server
docker compose logs api | grep SSH
```

10. Resumo das Variáveis de Ambiente

Variável	Obrigatória	Padrão	Descrição
DEPLOYMENT_MODE	Sim	self-hosted	self-hosted ou saas. Deve ser self-hosted para ativar admin panel.
POSTGRES_PASSWORD	Sim	—	Senha do PostgreSQL
JWT_SECRET	Sim	—	Secret para tokens de acesso (mín. 32 chars)
JWT_REFRESH_SECRET	Sim	—	Secret para tokens de refresh (mín. 32 chars)
FRONTEND_URL	Sim	http://localhost	URL pública da instância
CORS_ORIGINS	Não	FRONTEND_URL	Origens permitidas (separadas por vírgula)
GITHUB_CLIENT_ID	Sim*	placeholder	Client ID do GitHub OAuth
GITHUB_CLIENT_SECRET	Sim*	placeholder	Client Secret do GitHub OAuth
GOOGLE_CLIENT_ID	Não	placeholder	Client ID do Google OAuth
GOOGLE_CLIENT_SECRET	Não	placeholder	Client Secret do Google OAuth
S3_ENDPOINT	Não	—	Endpoint S3 para assets
S3_BUCKET	Não	—	Nome do bucket S3
GIT_STORAGE_PATH	Não	/data/repositories	Caminho dos repositórios Git
SSH_PORT	Não	2222	Porta do servidor SSH Git

*Pelo menos um provedor OAuth (GitHub ou Google) deve estar configurado corretamente para permitir login.

11. Suporte

- **Documentação da API:** <https://git.suaempresa.com/docs/api>
- **Issues:** [GitHub Issues](#)
- **Segurança:** Veja `SECURITY.md` para reportar vulnerabilidades